Smart City Interop Platform — Full Readme

> Plateforme SOC (Smart Operations Center) couvrant l'intégralité de la démonstration : services REST/SOAP/gRPC/GraphQL, orchestrateur SmartFlow, gateway Kong et tableau de bord React. Ce document rassemble toutes les informations nécessaires pour comprendre, exécuter et étendre le projet.

Table des matières

1. Vision & objectifs
2. Topologie & flux
3. Structure du dépôt
4. Cycle d'exécution
5. Composants détaillés
6. Contrats & interopérabilité
7. Sécurité & gateway
8. Stockage & données
9. Observabilité & résilience
10. Tests & qualité
11. Guides développeur
12. Pistes d'évolution
13. Historique & décisions

Vision & objectifs

- **Interop multi-protocoles** : prouver que REST, SOAP, gRPC et GraphQL peuvent coexister derrière une seule passerelle (Kong) tout en partageant une identité commune.
- **Orchestration métier** : SmartFlow (Node/Express) compose les services bas niveau, ajoute des retries, du cache Redis et des flux SSE.
- **Expérience citoyenne** : le client React fournit un point d'entrée unique pour la mobilité, la qualité de l'air, l'annuaire et les alertes.
- **Traçabilité des contrats** : tous les schémas (OpenAPI, WSDL, GraphQL SDL, proto gRPC) vivent dans `contracts/` et servent de source de vérité pour tests, clients et documentation.

Topologie & flux

```
[React SPA] —HTTP/S→ [Kong Gateway]
                      │  (JWT + x-api-key, TLS optionnel)
                      ├─ /api/smartflow/* → [SmartFlow Orchestrator]
                      │                        ├─ Mobility REST (Spring +
Postgres)
                      │                        ├─ AirQuality SOAP (JAX-WS)
                      │                        └─ Emergency gRPC (Spring Boot +
Postgres)
                      ├─ /api/directory → Directory GraphQL (Spring Boot +
Postgres)
```

```
                        └─ /api/mobility/* → Mobility REST (accès direct si besoin)

[Redis] ⇔ SmartFlow cache AQI        [Postgres] ⇔ Mobility bookings / Directory
services / Emergency alerts
```

- Ports internes : Mobility `8080`, AirQuality `8081`, Emergency gRPC `50051` (management HTTP `9090`), Directory GraphQL `8082`, SmartFlow `7000`, Kong `8000/8443`, Web dashboard `4173`.
- Flux temps réel : Emergency gRPC → SmartFlow SSE → Dashboard alertes.

Structure du dépôt

Chemin	Description
`client/web-dashboard/`	SPA React (Vite, TanStack Query, Playwright E2E).
`orchestrator/smartflow/`	Service Express orchestrant REST/SOAP/gRPC avec cache Redis (fallback mémoire).
`services/`	Services métiers (mobility REST, airquality SOAP, emergency gRPC Spring, directory GraphQL Spring).
`gateway/`	Configuration Kong (`kong.yml`, certificats TLS).
`contracts/`	OpenAPI, WSDL, GraphQL SDL, fichiers proto gRPC.
`infra/databases/database.sql`	Export consolidé des schémas/données (PostgreSQL multi-services).

Tout le contenu indispensable issu des documents d'architecture, de sécurité et de phases de livraison est condensé dans ce Full Readme pour éviter toute navigation supplémentaire.

Cycle d'exécution

1. **Pré-requis** : Docker Desktop (WSL2) démarré, ports 8000/8443/4173 libres.
2. **Build complet** : `docker compose build`.
3. **Démarrage** : `docker compose up` (ou `-d`). Attendre les logs `Started MobilityApplication`, `Started DirectoryServiceApplication` et `Started EmergencyServiceApplication`.
4. **Accès** :
   - Gateway HTTP: `http://localhost:8000`
   - Gateway HTTPS: `https://localhost:8443` (si `gateway/certs` présents)
   - Dashboard: `http://localhost:4173`
5. **Authentification** : ajouter `Authorization: Bearer <jwt>` et `x-api-key: citizen-api-key` sur chaque appel via Kong.
6. **Bascule dev ciblée** : `docker compose up -d --build web-dashboard` pour reconstruire uniquement le frontend après modifications.

Composants détaillés

1. Web Dashboard (`client/web-dashboard`)

- **Stack** : React + Vite + TypeScript, TanStack Query, Playwright.
- **Pages** :

    - *Dashboard AQI* (`GET /api/smartflow/metrics/overview`) — 3 colonnes (focus AQI, polluants Grand Tunis, alertes).
    - *Planificateur multimodal* (`POST /api/smartflow/trip-plan`) — formulaire zone/mode/stop, résultats + réservation.
    - *Annuaire & tickets* (GraphQL `/api/directory`) — filtres sur `citizenId`, mutations `createTicket`/`closeTicket`.
    - *Alertes temps réel* — SSE `/api/smartflow/alerts/stream` avec fallback `/alerts/recent`.
- **Sécurité UX** : écran de login centralisé, stockage JWT/API key dans `SessionContext`, injection automatique des en-têtes.
- **Scripts** : `npm run dev`, `npm run build`, `npm run test:e2e` (avec `RUN_E2E=1`).

2. SmartFlow Orchestrator (`orchestrator/smartflow`)

- **Endpoints** :
    - `POST /trip-plan` — combine SOAP AQI + REST mobility pour recommander un mode (règles basées sur l'AQI) et renvoie lignes/arrivées.
    - `GET /metrics/overview` — exécute en parallèle `fetchAQI`, `fetchLines`, `fetchArrivals`, `fetchRecentAlerts`.
    - `GET /alerts/recent` & `/alerts/stream` — proxy gRPC → SSE (heartbeat 15s, annulation propre des streams).
- **Clients** :
    - `airQualityClient` — enveloppe SOAP, parsing `fast-xml-parser`, retries `p-retry`, cache Redis 5 minutes (flag `stale`).
    - `mobilityClient` — Axios (timeout 2s, `AbortController`).
    - `emergencyClient` — `@grpc/grpc-js`, streaming.
- **Résilience** : middleware d'erreurs (`orchestration_failed`), logs `pino-http`, cache TTL configurable (`CACHE_DEFAULT_TTL`).

3. Gateway Kong (`gateway/kong`)

- **Routage** : `/api/mobility/*`, `/api/airquality`, `/api/emergency/*`, `/api/directory`, `/api/smartflow/*`.
- **Sécurité** : plugin `jwt` (HS256), `key-auth`, TLS optionnel (port 8443) via `gateway/certs`.
- **Consumers & secrets** :
    - `citizen-app` — clé API `citizen-api-key`, secret JWT `supersecretcitizen`.
    - `operator-app` — clé API `operator-api-key`, secret JWT `supersecretoperator`. Les deux profils ont une durée de vie maximale de 3600 s par jeton.

4. Domain Services (`services/`)

Service	Stack	Description
`mobility-rest`	Spring Boot 3 + PostgreSQL	Lignes/horaires/arrivées, réservations persistées (`BookingEntity`). Horaires générés dynamiquement (offset 6 min/stop), arrivées jitter ±1 min.
`airquality-soap`	Java JAX-WS	Opérations `GetAQI`, `GetPollutants`, `CompareZones`. Données en mémoire pour les zones du Grand Tunis.

Service	Stack	Description
`emergency-grpc`	Spring Boot 3 + gRPC + PostgreSQL	`SendAlert`, `GetRecentAlerts`, `SubscribeAlerts`. Alerts stockées en Postgres + diffusion streaming gRPC.
`directory-graphql`	Spring Boot 3 + GraphQL + PostgreSQL	Annuaire municipal + tickets citoyens migrés vers Postgres via Flyway + GraphQL Java.

5. Contracts (`contracts/`)

- `openapi/mobility-openapi.yaml` — source Swagger pour mobility.
- `soap/airquality.wsdl` — interface WSDL 1.1.
- `graphql/directory-schema.graphql` — schéma SDL importé par Apollo.
- `grpc/emergency.proto` — types `Alert`, `RecentRequest`, RPC `SendAlert`, etc.

Contrats & interopérabilité

- Chaque service consomme directement son fichier de contrat lors des builds (ex : `make proto`, `npm run codegen`).
- Les tests d'intégration recommandés :
  - REST → Postman/Newman basé sur l'OpenAPI.
  - SOAP → SoapUI en important `airquality.wsdl`.
  - GraphQL → `graphql-inspector`.
  - gRPC → `ghz`.

Sécurité & gateway

- **Auth obligatoire** : `Authorization: Bearer <JWT>` + `x-api-key: citizen-api-key` (ou `operator-api-key`).
- **JWT** : HS256 signé côté Kong. Pour générer un token de test côté PowerShell :

```
$header = '{"alg":"HS256","typ":"JWT"}'
$payload = '{"iss":"citizen-app","exp":' + ([int](Get-Date -UFormat %s) +
900) + '}'
function Encode([byte[]]$bytes) {
  return [Convert]::ToBase64String($bytes).TrimEnd('=') -replace '\+','-' -
replace '/','_'
}
$unsigned = (Encode([Text.Encoding]::UTF8.GetBytes($header))) + '.' +
(Encode([Text.Encoding]::UTF8.GetBytes($payload)))
$secret = 'supersecretcitizen'
$hmac = New-Object System.Security.Cryptography.HMACSHA256
([Text.Encoding]::UTF8.GetBytes($secret))
$signature =
Encode($hmac.ComputeHash([Text.Encoding]::UTF8.GetBytes($unsigned)))
$jwt = "$unsigned.$signature"
Write-Output $jwt
```

Remplacer `citizen` par `operator` pour l'autre consumer.

- **Exemple complet** :

```
curl https://localhost:8443/api/smartflow/metrics/overview \
  -H "x-api-key: citizen-api-key" \
  -H "Authorization: Bearer <jwt>"
```

- **TLS** : activer en plaçant `server.crt/server.key` dans `gateway/certs` (montés en lecture seule).
  1. Créer le dossier `gateway/certs`.
  2. Générer un certificat auto-signé :

```
openssl req -newkey rsa:2048 -nodes -keyout gateway/certs/server.key \
  -x509 -days 365 -out gateway/certs/server.crt -subj "/CN=localhost"
```

  3. S'assurer que `docker-compose.yml` monte ce dossier et que les variables `KONG_SSL_CERT` et `KONG_SSL_CERT_KEY` pointent dessus.
- **Client web** : gère l'injection d'en-têtes via `SessionContext` (pas de stockage dans le code).

Stockage & données

Système	Usage	Notes
PostgreSQL (`pg_mobility_data`)	Table `bookings` (réservations). `schema.sql` appliqué à chaque démarrage, données persistées via volume Docker.	
PostgreSQL (`pg_directory_data`)	Tables `directory_services`, `directory_tickets`, `service_hours`. Seeds Flyway, projections GraphQL.	
PostgreSQL (`pg_emergency_data`)	Tables `emergency_alerts`, `emergency_channels`. Historiques persistés + requêtes temps réel.	
Redis	Cache AQI (SmartFlow). Si indisponible, fallback mémoire avec TTL identique.	
In-memory	AirQuality SOAP dataset (zones Grand Tunis).	
`infra/databases/database.sql`	Export consolidé (Postgres multi services) pour audits/superviseurs.	

Observabilité & résilience

- **Logs** :
  - Kong → stdout JSON.
  - SmartFlow → `pino` (inclut `req.id`).
  - Emergency gRPC → Spring Boot (Logback + Micrometer).
  - Frontend → console filtrée (mode production : build Vite `preview`).
- **Retries/Timeouts** : `p-retry` (SmartFlow clients), `AbortController` (HTTP), heartbeat SSE 15s.

- **Cache** : TTL configuré via `CACHE_DEFAULT_TTL`, invalidation manuelle possible en supprimant la clé Redis.

Tests & qualité

Zone	Commandes
SmartFlow	`cd orchestrator/smartflow && npm test`
Directory Service	`cd services/directory-graphql && mvn test`
Emergency Service	`cd services/emergency-grpc && mvn test`
Web dashboard	`cd client/web-dashboard && npm run test:e2e` (set `RUN_E2E=1`, `PLAYWRIGHT_BASE_URL=http://localhost:4173`)
Mobility REST	`cd services/mobility-rest && mvn test` (optionnel)
Lint/Build	`npm run build` (frontend), `mvn package` (services Java)

Guides développeur

- **Variables critiques** :
  - SmartFlow: `MOBILITY_BASE_URL`, `AIRQUALITY_SOAP_URL`, `EMERGENCY_GRPC_URL`, `REDIS_URL`, `CACHE_DEFAULT_TTL`.
  - Directory Service: `DIRECTORY_DB_URL`, `DIRECTORY_DB_USERNAME`, `DIRECTORY_DB_PASSWORD`, `PORT`.
  - Emergency Service: `EMERGENCY_DB_URL`, `EMERGENCY_DB_USERNAME`, `EMERGENCY_DB_PASSWORD`, `GRPC_PORT`, `MANAGEMENT_HTTP_PORT`.
  - Web dashboard: `VITE_GATEWAY_URL`, `VITE_JWT`, `VITE_API_KEY`.
- **Hot reload ciblé** : lancer un service seul (`mvn spring-boot:run` côté Java, `npm run dev` pour SmartFlow/React) et pointer Kong/SmartFlow/React vers les URLs locales.
- **Rebuild sélectif Docker** : `docker compose up -d --build <service>`.
- **Database tips** :
  - Vérifier Postgres : `docker exec -it postgres-mobility psql -U mobility -c 'select * from bookings;'`.
  - Vérifier Directory : `docker exec -it postgres-directory psql -U directory -c 'select * from directory_services;'`.
  - Vérifier Emergency : `docker exec -it postgres-emergency psql -U emergency -c 'select * from emergency_alerts;'`.

Pistes d'évolution

1. **Intégration Directory dans SmartFlow** : ajouter un client GraphQL (`directoryClient.ts`) si l'orchestrateur doit exposer aussi les tickets.
2. **Observabilité avancée** : brancher Fluent Bit → Loki + Grafana pour centraliser les logs et SSE metrics.

3. **Auth OIDC** : remplacer le JWT statique par un fournisseur OpenID Connect (Keycloak/Azure AD) pour la gateway.
4. **Testing E2E automatisé** : brancher Playwright dans la CI (`RUN_E2E=1`) une fois un environnement preview stable disponible.
5. **Résilience data** : externaliser l'état Emergency (Redis ou Postgres) pour conserver l'historique au-delà d'un redémarrage.
6. **Internationalisation UI** : aujourd'hui ciblé Grand Tunis, prévoir un module i18n pour élargir la zone géographique.

Historique & décisions

- **Phase vision** : définition des quatre verticaux (mobilité, air, urgences, annuaire) et choix technos hétérogènes pour démontrer l'interop (Spring Boot REST/gRPC/GraphQL, JAX-WS SOAP).
- **Phase architecture** : adoption d'une gateway unique (Kong) avec politiques JWT + clé API, SmartFlow comme couche d'orchestration pour encapsuler SOAP/gRPC et exposer un REST unique.
- **Phase migration** : remplacement des anciens services Go/Apollo par `emergency-grpc` et `directory-graphql` Spring Boot + PostgreSQL afin d'unifier la persistence et la toolchain Maven.
- **Phase sécurité** : implémentation des consumers `citizen-app`/`operator-app`, génération de tokens HS256, ajout TLS optionnel et exigences de double facteur (JWT + x-api-key).
- **Phase client web** : refonte du login (cartouche centré), intégration des sections AQI/planification/annuaire/alertes et ajout du Trip Planner avec horaires/arrivées Mobility REST.
- **Phase ops** : ajout des exports `infra/databases/database.sql`, documentation des commandes Docker ciblées, introduction des caches Redis et des SSE heartbeat 15s pour éviter les timeouts proxy.

Ce *Full Readme* se veut la référence unique pour toute la plateforme SOC : architecture, exploitation, développement et perspectives futures.